

CONTENTS

Refuerzo — Representación Interna de Datos: Signo, Operar, IEEE 754	
Arquitectura de Computadoras — FING UdelaR · Guía de repaso dirigido (punto débil detectado)	
Parte 1 — Clasificar el signo en cada representación (n bits)	
Ejemplo resuelto — clasificar el mismo patrón en las 4 representaciones (8 bits)	
Ejercicio real — Parcial 26/09/2022, Pregunta 4 (representabilidad de -128 en 8 bits)	
Ejercicio real — Simulacro Primer Parcial 2024, Pregunta 3 (misma tira, 3 lecturas distintas)	
Parte 2 — Cómo operar (sumar/restar) en cada representación	
Ejercicio real — Parcial 5/10/2024, detección de overflow en C2 (extracto de código C del parcial)	
Parte 3 — IEEE 754: formato y suma paso a paso	
Ejemplo real 1 (verificado) — Parcial 25/09/2025, media precisión (exponentes cercanos, sin sorpresas)	
Ejemplo real 2 (verificado) — Examen 11/02/2026, simple precisión (exponentes MUY separados → absorción)	
Ejercicios de práctica (con solución, para autoevaluarte)	

Refuerzo — Representación Interna de Datos: Signo, Operar, IEEE 754

ARQUITECTURA DE COMPUTADORAS — FING UDELAR · GUÍA DE REPASO DIRIGIDO (PUNTO DÉBIL DETECTADO)

Esta guía se armó a partir de un punto débil concreto: clasificar signo en cada representación, operar (sumar/restar), y sobre todo la suma en IEEE 754. Todos los ejercicios de esta guía son de **tus propios parciales** (con solución oficial, verificada de forma independiente antes de incluirla acá).

Parte 1 — Clasificar el signo en cada representación (n bits)

Representación	Regla del signo	Cómo obtener la magnitud si es negativo	Rango representable	Representación del 0
Valor Absoluto y Signo (VA+S)	MSB: 0=positivo, 1=negativo	El resto de los bits (n-1) ya es la magnitud en binario puro	$-(2^{n-1}-1) \leq X \leq 2^{n-1}-1$	Dos representaciones: <code>0 0...0</code> y <code>1 0...0</code>
Complemento a 1 (C1)	MSB: 0=positivo, 1=negativo	Invertir todos los bits (incluido el de signo) → da la magnitud	$-(2^{n-1}-1) \leq X \leq 2^{n-1}-1$	Dos representaciones: <code>0 0...0</code> y <code>1 1...1</code>
Complemento a 2 (C2)	MSB: 0=positivo, 1=negativo	Invertir todos los bits y sumar 1 → da la magnitud	$-2^{n-1} \leq X \leq 2^{n-1}-1$	Una sola: <code>0 0...0</code>
Exceso/Desplazamiento (d)	No hay bit de signo — el patrón completo se lee como binario sin signo y se le resta el exceso d	Negativo si (valor binario puro) < d; valor real = (binario puro) - d siempre	$-d \leq X \leq 2^{n-1}-d$	Una sola: el patrón que da exactamente d (con $d=2^{n-1}$: <code>1 0...0</code>)

⚠ El error más común (y el que más confunde): aplicar la regla de C2 (invertir y sumar 1) sobre un número en VA+S, o asumir que “los bits después del signo son la magnitud directa” en C1/C2 (eso **solo** vale en VA+S). Cada representación tiene su propia regla — no hay atajo único.

Atajo útil para C2 (evita el paso de invertir+sumar1 si querés ir directo al valor): el bit de signo pesa -2^{n-1} y el resto de los bits pesan positivo normal. Ej. en 8 bits, el patrón `1 0110100` vale: $-128 + (0110100_2=52) = -76$.

EJEMPLO RESUELTO — CLASIFICAR EL MISMO PATRÓN EN LAS 4 REPRESENTACIONES (8 BITS)

Patrón: **10110100**

- **VA+S:** MSB=1 → negativo. Magnitud= $0110100_2=52 \rightarrow -52$.
- **C1:** MSB=1 → negativo. Invierto todo: $01001011_2=75 \rightarrow -75$.
- **C2:** MSB=1 → negativo. Invierto (01001011) y sumo 1 → $01001100_2=76 \rightarrow -76$.
- **Exceso 128:** binario puro= $10110100_2=180 \rightarrow 180-128 \rightarrow +52$.

(Verificación cruzada C2: atajo del bit de signo: $-128+52=-76 \checkmark$ coincide.)

EJERCICIO REAL — PARCIAL 26/09/2022, PREGUNTA 4 (REPRESENTABILIDAD DE -128 EN 8 BITS)

a) Representar **-126 en 8 bits:** $126_{10}=1111110_2$.

- VA+S: `1 1111110`
- C2: invierto (10000001) y sumo 1 → `1 0000010`
- Exceso 127: $-126+127=1 \rightarrow 00000001$

b) ¿Es representable **-128 en cada una?**

- VA+S: **No.** Rango es $-(2^7-1)=-127$ hasta 127. $-128 < -127$, fuera de rango.
- C2: **Sí.** `10000000` (caso límite exacto: $-2^7=-128$).
- Exceso 127: **No.** Rango es -127 hasta 128. $-128 < -127$, fuera de rango.

(Este ejercicio es la clave para entender **por qué C2 es asimétrico** — tiene un negativo más que positivos, exactamente el que VA+S y Exceso no pueden representar.)

EJERCICIO REAL — SIMULACRO PRIMER PARCIAL 2024, PREGUNTA 3 (MISMA TIRA, 3 LECTURAS DISTINTAS)

Tira: `1100100011000000` (16 bits)

a) Como Punto Flotante Media Precisión (1 signo, 5 exponente $d=15$, 10 mantisa): signo=1, exponente= $10010_2=18$ (real= $18-15=3$), mantisa= $0011000000 \rightarrow -1,0011_2 \cdot 2^3 = -1001,1_2 = -9,5$

b) Como Punto Fijo, C2, 8 bits de parte fraccionaria: Descomplementar (invertir+1):

`0011011101000000`. Los 8 bits menos significativos son la parte fraccionaria. $\rightarrow -110111,01_2 = -55,25$

c) Como Exceso $d=2^{15}$: El patrón empieza con 1, así que **es mayor que el desplazamiento** (positivo en la interpretación de exceso). Resto d : $1100100011000000_2 - 1000000000000000_2 = 0100100011000000_2 = 18624$

(Fijate que la MISMA tira de bits da tres valores completamente distintos según qué representación asumas — por eso el enunciado siempre tiene que aclarar qué formato es. Este es el ejercicio perfecto para practicar “leer el mismo patrón con reglas distintas” sin mezclarlas.)

Parte 2 — Cómo operar (sumar/restar) en cada representación

- **C2 es la única donde sumás/restás como binario normal, sin distinguir signos** — esa es justamente la razón por la que casi todo el hardware real usa C2. Se ignora el carry que sobra del bit más significativo (para n bits fijos).
- **Overflow en C2:** comparar el carry que *entra* al bit de signo contra el carry que *sale* de él — si son distintos, hay overflow. Equivalente (más fácil de chequear con los signos): si sumás dos números del **mismo signo** y el resultado da signo **opuesto**, hubo overflow.
- **VA+S** no se suma directo: si los signos son iguales, sumás magnitudes y mantenés el signo; si son distintos, restás la magnitud menor a la mayor y el resultado toma el signo del operando de mayor magnitud.
- **C1** es como C2 pero con una corrección extra: si sobra un carry del bit más significativo, hay que sumarlo de vuelta al resultado (“end-around carry”).

EJERCICIO REAL — PARCIAL 5/10/2024, DETECCIÓN DE OVERFLOW EN C2 (EXTRACTO DE CÓDIGO C DEL PARCIAL)

```
if (num1 * num2 > 0) {           // los números tienen el mismo signo
    if (num1 * resultado < 0) { // y el resultado da signo opuesto
        flag_V = 1;           // hay overflow
    }
}
```

Esto es exactamente la regla de arriba: incluso a nivel de implementación real, la detección de overflow en C2 se reduce a comparar signos de operandos vs. signo del resultado — no hace falta mirar bit a bit el carry si ya tenés los valores con signo disponibles.

Parte 3 — IEEE 754: formato y suma paso a paso

Formato simple precisión (32 bits): 1 signo | 8 exponente (exceso 127) | 23 mantisa, con 1 implícito → $\text{valor} = \pm 1, \text{mantisa} \times 2^{\text{exp}-127}$
Formato media precisión (16 bits): 1 signo | 5 exponente (exceso 15) | 10 mantisa → $\text{valor} = \pm 1, \text{mantisa} \times 2^{\text{exp}-15}$

El signo en IEEE754 es el más simple de los cuatro (bit más alto, 0=positivo/1=negativo, sin ninguna otra regla) — lo que complica es el **signo del resultado** al sumar.

Algoritmo de suma A+B:

1. **Igualar exponentes:** el número de exponente menor desplaza su mantisa a la derecha tantas posiciones como diferencia de exponentes haya.
2. **Operar mantisas** (con el 1 implícito restaurado): signos iguales → sumar; signos distintos → restar la menor de la mayor (en valor absoluto).
3. **Signo del resultado:** si sumaste, conserva el signo común. Si restaste, toma el signo del operando de **mayor magnitud**.
4. **Normalizar:** ajustar el punto y el exponente hasta que la mantisa vuelva a empezar con “1,”.
5. **Redondear** si sobran bits más allá de los disponibles.

EJEMPLO REAL 1 (VERIFICADO) — PARCIAL 25/09/2025, MEDIA PRECISIÓN (EXPONENTES CERCANOS, SIN SORPRESAS)

A: 1100 0000 0100 0000, B: 0011 1111 1000 0000

- **A:** signo=1, exponente= $10000_2=16$ (real= $16-15=1$), mantisa= $0001000000 \rightarrow -1,0001_2 \cdot 2^1 = -10,001_2 = -2,125$
- **B:** signo=0, exponente= $01111_2=15$ (real=0), mantisa= $1110000000 \rightarrow 1,111_2 = 1,875$

Signos distintos → resto la menor magnitud de la mayor:

```

10,001   (2,125)
- 01,111 (1,875)
= 00,010 (0,25)

```

Signo del resultado = signo del operando de mayor magnitud (A, negativo) → **C = -0,25**

Normalizando: $-0,25 = -1,0_2 \cdot 2^{-2}$. Exponente desplazado = $-2+15=13=01101_2$.

C = 1011 0100 0000 0000

EJEMPLO REAL 2 (VERIFICADO) — EXAMEN 11/02/2026, SIMPLE PRECISIÓN (EXPONENTES MUY SEPARADOS → ABSORCIÓN)

A = $1,0000001_2 \cdot 2^{-126}$ (el exponente normalizado más chico posible), B = $-1,1111111_2 \cdot 2^{-1}$

- **A:** signo=0, exp=-126→desplazado=1= 00000001, mantisa= 000000100000000000000000 → 0 00000001 000000100000000000000000
- **B:** signo=1, exp=-1→desplazado=126= 01111110, mantisa= 111111100000000000000000 → 1 01111110 111111100000000000000000

Para sumar, hay que igualar exponentes: la diferencia es $-1 - (-126) = 125$ posiciones. Desplazar la mantisa de A 125 lugares a la derecha la deja completamente fuera del rango de los 23+1 bits significativos disponibles — **A se pierde por completo** frente a la precisión limitada de B.

Resultado: la suma da exactamente B → `1 01111110 111111100000000000000000` (sin cambios).

Por qué importa este ejemplo: es el caso donde un número extremadamente chico (2^{-126}) se “absorbe” al sumarlo con uno mucho más grande (2^{-1}) — la resta/suma en punto flotante **no es exacta** cuando las magnitudes difieren demasiado. Es un fenómeno real (mismo problema que causa errores de precisión acumulados en cálculo numérico) y aparece seguido en exámenes como este.

Ejercicios de práctica (con solución, para autoevaluarte)

1. Clasificá el patrón de 8 bits `01101011` en VA+S, C1, C2 y Exceso 128. (Solución: VA+S=+107, C1=+107, C2=+107, Exceso128=-21 — como el bit más significativo es 0, en VA+S/C1/C2 siempre da positivo y coincide; en Exceso el patrón “0...” siempre da negativo, exactamente al revés de la intuición del bit de signo.)
2. Sumá en C2 (8 bits): `01001101 + 00110101`. Indicá si hay overflow. (Solución: `01001101(+77)+00110101(+53)=10000010`; signos de entrada iguales (+,+) pero resultado tiene MSB=1 (negativo) → **hay overflow**, el resultado de 130 no entra en 8 bits con signo.)
3. Sumá en IEEE754 media precisión: A= `0100 0000 0000 0000` (+2,0), B= `1011 1000 0000 0000` (revisá vos el signo/exponente/mantisa de B antes de sumar). Prestá especial atención a si los signos son iguales o distintos antes de decidir si sumás o restás las mantisas.

Si querés, en la próxima sesión resolvemos el ejercicio 3 juntos paso a paso (con la metodología de “un ejercicio a la vez, marco el error exacto si lo hay”) para terminar de consolidar el algoritmo de suma.